

Repaso: Chat, Cowork, Code y los modelos que hay detrás

Claude Certified Developer, Foundation (CCDV-F) · Masterclass UST

ES Sesión 4 de 12 · Notas en español (es-ES)

La sesión 4 (10 de agosto de 2026) arrancó con un repaso de las tres superficies de cliente: **Claude Chat** (claude.ai), **Claude Cowork** (escritorio) y **Claude Code** (agente de terminal / IDE). El formador, Vijay, usó ese mapa para seguir generando de extremo a extremo una app de gestión de tareas, todavía sin escribir el código de la aplicación a mano.

INTERFAZ DE AULA FRENTE A LA DOCUMENTACIÓN VIGENTE

Las etiquetas, los nombres de modelo (familias Haiku / Sonnet / Opus) y los menús slash de Claude Code de la grabación de agosto de 2026 cambiarán. Para el examen y para entrega, priman docs.claude.com y la **documentación de Claude Code**.

Los tres productos en una frase cada uno

El repaso no son tres apps sueltas. Comparten familias de modelo y muchos patrones de memoria (CLAUDE .md, skills). Difieren en *dónde ocurre el trabajo* y cuánta agencia tiene el cliente sobre vuestro ordenador.

Producto	Énfasis del formador	Uso típico en esta cohorte
Claude Chat (claude.ai)	Pregunta–respuesta; elegís Haiku, Sonnet u Opus (o el equivalente vigente) y obtenéis respuesta.	Exploración, borradores, preguntas que no necesitan un repositorio.
Claude Cowork	Cliente de escritorio: carpetas, tareas repetitivas, plantillas y (en esta sesión) Design / wireframes.	Organizar carpetas de trabajo; generar artefactos que luego alimentan Code.
Claude Code	Programación agentic: plan, herramientas, sandbox, apps con forma de producción a partir de prompts, skills y memoria.	Generación full-stack, backends OpenAPI-first, frontales con MCP.

Cowork no es «Chat con una ventana más bonita». Se parece más a un agente que puede trabajar sobre carpetas locales. Claude Code es la misma idea apuntada a un repositorio: un bucle de agente que lee, escribe, ejecuta comandos y verifica. Chat sigue siendo la superficie más ligera: preguntáis, el modelo responde, y no posee el árbol del proyecto.

Modelos: Haiku, Sonnet, Opus — y por qué el nombre no es el examen

En el repaso el formador nombró las tres familias (Haiku, Sonnet y Opus). El punto didáctico es la *escalera*, no un SKU congelado: modelo más barato/rápido para ediciones rutinarias, un intermedio por defecto para la mayor parte del código, y uno más pesado cuando el plan o la arquitectura piden más razonamiento. En

Claude Code se cambia con `/model` (módulo 05). Gana la disponibilidad de la organización: solo veis los modelos a los que el espacio de trabajo tiene derecho.

- Las mismas familias pueden estar detrás de Chat, Cowork y Code; el *cliente* cambia el acceso a herramientas y la persistencia, no la idea de «qué cerebro».
- Una sesión en directo puede mostrar un valor por defecto del tipo «Sonnet 5»; no memoricéis esa cadena. Memorizad: el modelo más pequeño que aún haga el trabajo, y comprobad la página de modelos antes del examen.
- El coste en tokens sigue la elección de modelo y cuánto adjuntáis (carpetas enteras frente a un solo YAML OpenAPI).

Memoria universal: CLAUDE.md frente a skills

El formador trató CLAUDE . md y las skills como patrones *universales*: valen para Cowork, Code y, donde el producto lo permita, flujos cercanos a Chat. Un alumno preguntó la diferencia; la respuesta es la que hay que llevar al examen.

Mecanismo	Cuándo se aplica	Cómo se invoca
CLAUDE.md (memoria)	Se añade por defecto a las peticiones que el cliente hace al modelo.	No se llama con slash. Va en la raíz del repo; ficheros extra opcionales en frontend/backend.
Skill (SKILL.md en una carpeta con nombre)	Solo cuando esa skill está en juego.	Comando slash (p. ej. <code>/business-analyst</code> , <code>/architect</code>), salvo que la invocación automática siga activa.

CLAUDE . md es instrucción de sistema permanente: código listo para producción, buenas prácticas del lenguaje, responsabilidad única, «para todos los diagramas usad Mermaid», etc. Ficheros anidados permiten normas distintas de backend y frontend. Las skills codifican un *rol* (analista de negocio, arquitecto, diseñador OpenAPI, ingeniero frontend). El cuerpo de la skill se escribe en SKILL . md (en mayúsculas) dentro de su carpeta; una vez definida, Claude Code expone el slash correspondiente.

INVOCACIÓN AUTOMÁTICA DE SKILLS

Por defecto Claude puede elegir una skill si vuestro texto encaja (SRS, «business analyst», etc.). Para forzar el uso solo con slash, activad la propiedad que desactiva la invocación por el modelo (*disable model invocation* a true, en la formulación del formador). En un bucle agentic las skills también pueden entrar sin que escribáis el slash: es intencionado, no un fallo.

Roles de SDLC como skills, no como modelos extra

La demo siguió la app de tareas como si un SDLC real estuviera cubierto por roles de Claude: ventas/stakeholders producen BRD/SRS con una skill de analista; los arquitectos producen HLD, diagramas de componentes y secuencia, OpenAPI; los desarrolladores consumen esos artefactos. Vosotros seguís revisando y publicando (PDF, Confluence). Claude redacta; las personas son dueñas del documento.

1. Skill de requisitos: dentro/fuera de alcance, login frente a registro, dashboard, CRUD de tareas.

2. Skill de arquitecto: pensar como arquitecto de solución; HLD, despliegue, flujo de datos, requisitos no funcionales.
3. Preferid Mermaid (u otro diagrama en texto) frente a JPEG/SVG: menos tokens, se edita como código, GitHub y Confluence lo renderizan.
4. Los mismos slash funcionan en la UI de Claude en VS Code y en la sesión de terminal `claude`; la vista previa Mermaid en el IDE necesita una extensión, GitHub no.

El bucle de cinco fases (Chat, Cowork y Code)

Ante una tarea sustancial en Cowork o Code, el formador describió un patrón de cinco pasos que también llamó ejecución al estilo chain-of-thought: **aclarar → planificar → ejecutar → verificar → entregar**. Ejemplo: «organiza mi carpeta Descargas por tipo de fichero». La ambigüedad genera preguntas (¿conservar nombres? ¿duplicados?). Luego un plan (escanear, clasificar imágenes/documentos/archivos), los movimientos, una comprobación de que no se ha perdido nada, y un resumen.

- La verificación (revisión de código limpio, regenerar tests) es cara. Muchos equipos ponen en la memoria «no verifiques automáticamente el código generado» y prueban en local.
- La secuencia se puede acotar en CLAUDE.md («terminad los controladores antes que los servicios»).
- Leed siempre los comandos destructivos. El consentimiento es vuestro; Anthropic no asume un borrado que hayáis aprobado.
- La técnica de prompting llamada chain of thought se deja para módulos posteriores; aquí es el *comportamiento* del bucle del agente, no un flag de API.

Qué llevar a los capítulos siguientes

El resto del módulo 04 deja el repaso de GUI y se centra en APIs design-first y MCP. Si solo retenéis tres hechos: (1) Chat / Cowork / Code son una familia con distinta agencia; (2) CLAUDE.md es memoria siempre activa, las skills son paquetes de rol; (3) el bucle es aclarar–planificar–ejecutar–verificar–entregar, y la verificación es un impuesto de tokens que podéis bajar.

Documentación oficial

• Inicio de Claude Docs

<https://docs.claude.com/en/home>

Cuando una etiqueta del aula ya no coincida con el producto.

• Modelos

<https://docs.claude.com/en/docs/about-claude/models>

Nombres vigentes de Haiku / Sonnet / Opus y ventanas de contexto.

• Visión general de Claude Code

<https://code.claude.com/docs/en/overview>

Agente de terminal e IDE de este módulo.

• Soporte de Cowork / escritorio

<https://support.claude.com/>

Cliente de escritorio, Design y acceso a la cuenta.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.